Ejercicio Demostrativo

Tabla hash — Unidad 3

Auxiliar: Sergio Sebastian Sandoval Ruiz

Tiempo estimado: 60 min

5 minutos para Respeto y 55 minutos de desarrollo académico

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas
Segundo Semestre 2026
Laboratorio de Estructuras de Datos C

1. Marco Formativo

1.1 Valores

Respeto	Escuchar las propuestas y los ritmos de aprendizaje de los compañeros. Durante la demostración, comparar estrategias de colisión sin descalificar, dar crédito a las ideas ajenas y explicar los errores con argumentos.

1.2 Competencias

Tipo de competencia	Redacción
Competencia general	Implementa estructuras de datos lineales y no lineales mediante programación orientada a objetos para desarrollar soluciones eficientes.
Competencia específica	Construye una tabla hash con encadenamiento en Java, implementa inserción, consulta, actualización y eliminación, y verifica colisiones, factor de carga y rehashing. Contrasta su funcionamiento con el sondeo lineal.

2. Palabras Clave

Tabla hash: Estructura que asocia claves con valores y calcula una posición de acceso mediante una función de dispersión.

Clave y valor: Identificador usado en una consulta y dato asociado a ese identificador.

Función hash e índice: Transformación de la clave y reducción al intervalo de posiciones de la tabla.

Colisión: Coincidencia de índice entre dos claves diferentes.

Encadenamiento: Colección de entradas por cubeta para resolver colisiones.

Sondeo lineal: Exploración de posiciones consecutivas del arreglo tras una colisión.

Factor de carga: Relación n/m entre asociaciones activas y capacidad.

Rehashing: Reubicación de entradas al cambiar la capacidad de la tabla.

Marca de borrado: Estado que permite continuar una búsqueda por una posición eliminada en direccionamiento abierto.

3. Resumen

El ejercicio corresponde a la sesión de Tabla Hash de la unidad 3. El auxiliar implementa en Java un diccionario de claves enteras y valores de texto, provoca colisiones deliberadas y verifica que cada operación conserve las asociaciones correctas. También muestra por qué el sondeo lineal necesita distinguir una celda vacía de una celda borrada.

Al finalizar, el estudiante debe explicar cómo se obtiene el índice, por qué dos claves distintas pueden compartirlo y cómo cambian la capacidad y la distribución después de un rehashing. La demostración utiliza arreglos y nodos propios; no delega la estructura a HashMap.

4. Contenido

4.1 DESCRIPCIÓN DEL EJERCICIO

Construir `TablaHash.java` con listas enlazadas por cubeta y ejecutar `Main.java`. Usar capacidad inicial 7 y las claves 10, 17, 24 y 31, que comparten residuo 3. Actualizar una clave, eliminar otra y agregar entradas hasta activar el crecimiento a capacidad 17.

4.2 OBJETIVO DE LA DEMOSTRACIÓN

> Demostrar las operaciones de un diccionario hash, manteniendo una única entrada por clave y un contador coherente. Relacionar la ejecución del código con las trazas de las cubetas y justificar el manejo de colisiones y el costo de las operaciones.

4.3 RECURSOS NECESARIOS

- Java JDK 11 o superior, editor o IDE y terminal con javac y java.
- Proyector o pantalla compartida y diapositivas de Tabla Hash de la unidad 3.
- Dos archivos de trabajo: `TablaHash.java` y `Main.java`; capacidad inicial 7.
- Pizarrón o papel para dibujar cubetas, enlaces y secuencias de sondeo.

Distribución del tiempo

Actividad	Minutos
Respeto y acuerdos de participación	5
Conceptos y función de dispersión	15
Implementación con encadenamiento	20
Pruebas en consola	10
Contraste con sondeo lineal	5

Actividad	Minutos
Resultados y cierre	5

4.4 DESARROLLO PASO A PASO

El auxiliar muestra primero la traza y después el código que la produce. Los archivos se preparan en el mismo directorio. Las claves son enteras y los valores de texto no pueden ser null; así, get puede devolver null de manera inequívoca cuando la clave está ausente.

Paso 1 — Respeto y participación (5 min)

Pedir dos propuestas para guardar registros que comparten índice. Escuchar ambas sin interrumpir y comparar sus reglas con una misma secuencia. Acordar que una discrepancia se resuelve mediante la traza y las condiciones del algoritmo, manteniendo un trato respetuoso.

Paso 2 — Conceptos y función de dispersión (15 min)

Definir capacidad m, cantidad n y cubeta. Para claves no negativas, usar h(k) = k mod m. Con m = 7, las claves 10, 17, 24 y 31 generan índice 3. La colisión no permite sobrescribir una clave distinta: hay que comparar la clave almacenada.

Clave	Cálculo	Índice
10	10 mod 7	3
17	17 mod 7	3
24	24 mod 7	3
31	31 mod 7	3

La implementación inserta cada nuevo nodo al inicio de la lista. Después de las cuatro inserciones, la cubeta 3 contiene 31 → 24 → 17 → 10. Ese orden físico es válido aunque difiera del orden de llegada.

Consultar 24 recorre 31 y luego 24. Actualizar 17 cambia su valor sin crear otro nodo y sin aumentar n. Eliminar 10 retira el último nodo de esa lista y deja n = 3.

Qué observar:

- Una misma cubeta conserva varias claves distintas y sus valores correctos.
- La comparación de igualdad, y no solo el índice, identifica la asociación.
- El factor de carga inicial es 4/7, aproximadamente 0.571.
- Math.floorMod permite índices válidos también para claves negativas.

Paso 3 — Implementación (20 min)

Crear `TablaHash.java`. La primera parte define el nodo, calcula el índice e implementa put y get. Una clave existente se actualiza antes de comprobar el crecimiento. Las claves nuevas se agregan al inicio de su cubeta.

java

```java
public class TablaHash {
    private static class Nodo {
        int clave;
        String valor;
        Nodo siguiente;
        Nodo(int clave, String valor, Nodo siguiente) {
            this.clave = clave;
            this.valor = valor;
            this.siguiente = siguiente;
        }
    }
    private Nodo[] cubetas;
    private int cantidad;
    public TablaHash(int capacidad) {
        if (capacidad < 1)
            throw new IllegalArgumentException("Capacidad invalida");
        cubetas = new Nodo[capacidad];
    }
    private int indice(int clave) {
        return Math.floorMod(clave, cubetas.length);
```

```
    }
    public void put(int clave, String valor) {
        if (valor == null)
            throw new IllegalArgumentException("Valor nulo");
        int i = indice(clave);
        for (Nodo p = cubetas[i]; p != null; p = p.siguiente) {
            if (p.clave == clave) {
                p.valor = valor;
                return;
            }
        }
        if (cantidad + 1 > cubetas.length * 0.70)
            rehash(cubetas.length * 2 + 3);
        i = indice(clave);
        cubetas[i] = new Nodo(clave, valor, cubetas[i]);
        cantidad++;
    }
    public String get(int clave) {
        for (Nodo p = cubetas[indice(clave)];
             p != null; p = p.siguiente) {
            if (p.clave == clave) return p.valor;
        }
        return null;
    }
```

Continuación de TablaHash.java:

Agregar las líneas siguientes inmediatamente después de la primera parte. remove conserva el enlace anterior para retirar la cabeza o un nodo interior. rehash guarda el siguiente nodo antes de modificar sus enlaces.

java

```
    public String remove(int clave) {
        int i = indice(clave);
```

```
        Nodo anterior = null;
        Nodo actual = cubetas[i];
        while (actual != null) {
            if (actual.clave == clave) {
                if (anterior == null)
                    cubetas[i] = actual.siguiente;
                else anterior.siguiente = actual.siguiente;
                cantidad--;
                return actual.valor;
            }
            anterior = actual;
            actual = actual.siguiente;
        }
        return null;
    }
    private void rehash(int nuevaCapacidad) {
        Nodo[] anteriores = cubetas;
        cubetas = new Nodo[nuevaCapacidad];
        for (Nodo cabeza : anteriores) {
            Nodo actual = cabeza;
            while (actual != null) {
                Nodo siguiente = actual.siguiente;
                int i = indice(actual.clave);
                actual.siguiente = cubetas[i];
                cubetas[i] = actual;
                actual = siguiente;
            }
        }
    }
    public int size() { return cantidad; }
    public int capacidad() { return cubetas.length; }
```

```
    public double carga() {
        return (double) cantidad / cubetas.length;
    }
}
```

El tamaño no cambia durante rehash: se trasladan las asociaciones existentes. La política didáctica crece antes de que una nueva inserción supere una carga de 0.70. La fórmula de crecimiento 2m + 3 lleva la capacidad inicial de 7 a 17; no pretende generar siempre números primos.

La capacidad fija del ejemplo es pequeña para observar colisiones. En producción, la función de dispersión y la política de capacidad deben evaluarse con las claves reales.

Paso 4 — Pruebas en consola (10 min)

Crear `Main.java` en el mismo directorio. Ejecutar las operaciones en el orden mostrado. Primero hay cuatro asociaciones; actualizar no cambia ese número. Después de borrar 10 e insertar 38 vuelve a haber cuatro. Insertar 45 dispara el crecimiento.

java

```
public class Main {
    public static void main(String[] args) {
        TablaHash tabla = new TablaHash(7);
        tabla.put(10, "Ana");
        tabla.put(17, "Luis");
        tabla.put(24, "Eva");
        tabla.put(31, "Carlos");
        System.out.println("Buscar 24: " + tabla.get(24));
        tabla.put(17, "Luis actualizado");
        System.out.println("Actualizar 17: " + tabla.get(17));
        System.out.println("Tamano tras actualizar: " + tabla.size());
        System.out.println("Eliminar 10: " + tabla.remove(10));
        System.out.println("Buscar 10: " + tabla.get(10));
        tabla.put(38, "Marta");
        tabla.put(45, "Pedro");
        System.out.println("Capacidad final: " + tabla.capacidad());
        System.out.println("Tamano final: " + tabla.size());
```

```
        System.out.println("Buscar 31: " + tabla.get(31));
        System.out.println("Buscar 45: " + tabla.get(45));
        tabla.put(-1, "Negativo");
        System.out.println("Buscar -1: " + tabla.get(-1));
    }
}
```

Compilar y ejecutar:

```
javac TablaHash.java Main.java
java Main
```

Salida esperada:

```
Buscar 24: Eva
Actualizar 17: Luis actualizado
Tamano tras actualizar: 4
Eliminar 10: Ana
Buscar 10: null
Capacidad final: 17
Tamano final: 5
Buscar 31: Carlos
Buscar 45: Pedro
Buscar -1: Negativo
```

La última inserción agrega una sexta asociación después de imprimir el tamaño final 5. La clave −1 se ubica correctamente porque el índice se normaliza; no se utiliza el valor absoluto de la clave.

Paso 5 — Sondeo lineal (5 min)

Contrastar la implementación con otra estrategia sin modificar su código. En sondeo lineal se usa $h_i(k) = (h(k) + i) \bmod m$ y cada entrada ocupa una celda del arreglo. Repetir la secuencia 10, 17, 24 y 31 con capacidad 7.

Estado	Índice 3	Índice 4	Índice 5	Índice 6
Después de insertar	10	17	24	31

Estado	Índice 3	Índice 4	Índice 5	Índice 6
Después de eliminar 17	10	BORRADO	24	31
Después de insertar 38	10	38	24	31

Los índices 0, 1 y 2 permanecen vacíos. Buscar 24 debe continuar por el índice 4 aunque esté marcado BORRADO. Si se marcara VACÍO, la búsqueda se detendría antes de llegar a 24 y respondería incorrectamente.

Para reutilizar un borrado al insertar, se recuerda la primera marca, pero se continúa buscando por si la clave ya existe más adelante. Así una actualización no crea un duplicado. Limitar el recorrido a m posiciones evita ciclos infinitos.

Factor de carga y rehashing:

En el programa con encadenamiento, antes de insertar 45 existen las claves 17, 24, 31 y 38. Una quinta entrada produciría 5/7 ≈ 0.714, por lo que se reconstruye con m = 17. No se copian los índices anteriores: se vuelve a calcular el destino de cada clave.

Clave activa	Índice con m igual a 17
17	0
24	7
31	14
38	4
45	11

La carga pasa a 5/17 ≈ 0.294. Tras agregar −1, la cantidad es 6 y su índice es 16. En encadenamiento se eliminan enlaces; las marcas BORRADO pertenecen al ejemplo de direccionamiento abierto.

Observaciones Importantes

- No incrementar cantidad cuando put encuentra una clave existente.
- Al eliminar, distinguir la cabeza de la cubeta de un nodo interior; una ausencia no cambia el tamaño.
- Durante rehash, guardar actual.siguiente antes de reubicar actual. No volver a contar los nodos trasladados.
- Una función hash de diccionario no proporciona por sí sola almacenamiento seguro de contraseñas.
- O(1) es un costo esperado bajo buena dispersión y carga controlada; no es una garantía para todas las entradas.

4.5 Tabla Comparativa

TABLA COMPARATIVA

Operación	Encadenamiento	Sondeo lineal
Guardar colisiones	Nodos en una misma cubeta	Otra posición del arreglo
Buscar	Comparar claves de la lista	Seguir el sondeo de la clave
Eliminar	Reconectar enlaces	Marca o estrategia de reinserción
Carga	Puede superar 1	Se necesita espacio libre
Costo esperado	O(1) con carga acotada	O(1) con carga controlada
Peor caso de consulta	O(n)	O(n) con capacidad proporcional a n

4.6 Resultado Esperado

El estudiante debe observar colisiones sin pérdida de datos, una actualización que conserva el tamaño, un borrado correcto y un crecimiento que conserva todas las asociaciones activas. Una reconstrucción cuesta O(n) esperado en esta política; con crecimiento geométrico, la inserción tiene costo esperado amortizado constante bajo los supuestos de dispersión.

Verificación	Resultado
Consulta antes del borrado	get(24) devuelve Eva
Actualización de 17	Valor nuevo y size igual a 4
Borrado de 10	remove devuelve Ana y get devuelve null
Crecimiento tras insertar 45	Capacidad 17 y cinco asociaciones
Clave negativa	get(-1) devuelve Negativo

4.7 Aplicación Práctica

Una tabla hash puede asociar carnés con estudiantes, códigos con productos o palabras con frecuencias. Favorece consultas por igualdad. Si se necesitan intervalos o un orden por clave, debe considerarse una estructura ordenada. En una tabla de frecuencias, la actualización de una palabra existente no agrega otra clave.

Paso 6 — Resultados y cierre (5 min)

Pedir al grupo que explique por qué 10 y 17 pueden coexistir, por qué actualizar no aumenta el tamaño y por qué cambiar la capacidad exige reubicar las entradas. Comparar las respuestas con respeto y cerrar mostrando que eliminar una entrada no debe impedir encontrar otras claves que colisionaron.

5. Recomendaciones al siguiente tutor

Dificultades previstas:

- La principal confusión es tratar la coincidencia de índice como igualdad de clave. También conviene anticipar el caso de eliminación de cabeza y la modificación del enlace siguiente durante rehash. Dibujar la lista antes de cada cambio ayuda a comprobar el procedimiento.

Recomendaciones para futuras sesiones:

- Preparar los dos archivos y las llamadas de prueba antes de la clase. Escribir en vivo los métodos centrales y mostrar el resto cuando el tiempo sea limitado.
- Mantener las claves 10, 17, 24 y 31 para que las colisiones sean visibles. Añadir una clave negativa y una consulta ausente.

- Separar el borrado por enlaces de las marcas del sondeo lineal; cada estrategia tiene sus propias reglas.
- Pedir una justificación de los contadores y comprobar que cada clave siga accesible después del crecimiento.

6. Referencias

1. Sedgewick, R., y Wayne, K. Algorithms, 4th edition. Sección 3.4, Hash Tables. https://algs4.cs.princeton.edu/34hash/
2. Morin, P. Open Data Structures. Capítulo 5, Hash Tables. https://opendatastructures.org/newhtml/ods/latex2/hashing.html
3. Material de clase. Tabla Hash, unidad 3. Laboratorio de Estructuras de Datos, sección C, Escuela de Ingeniería en Ciencias y Sistemas, USAC, 2026.